

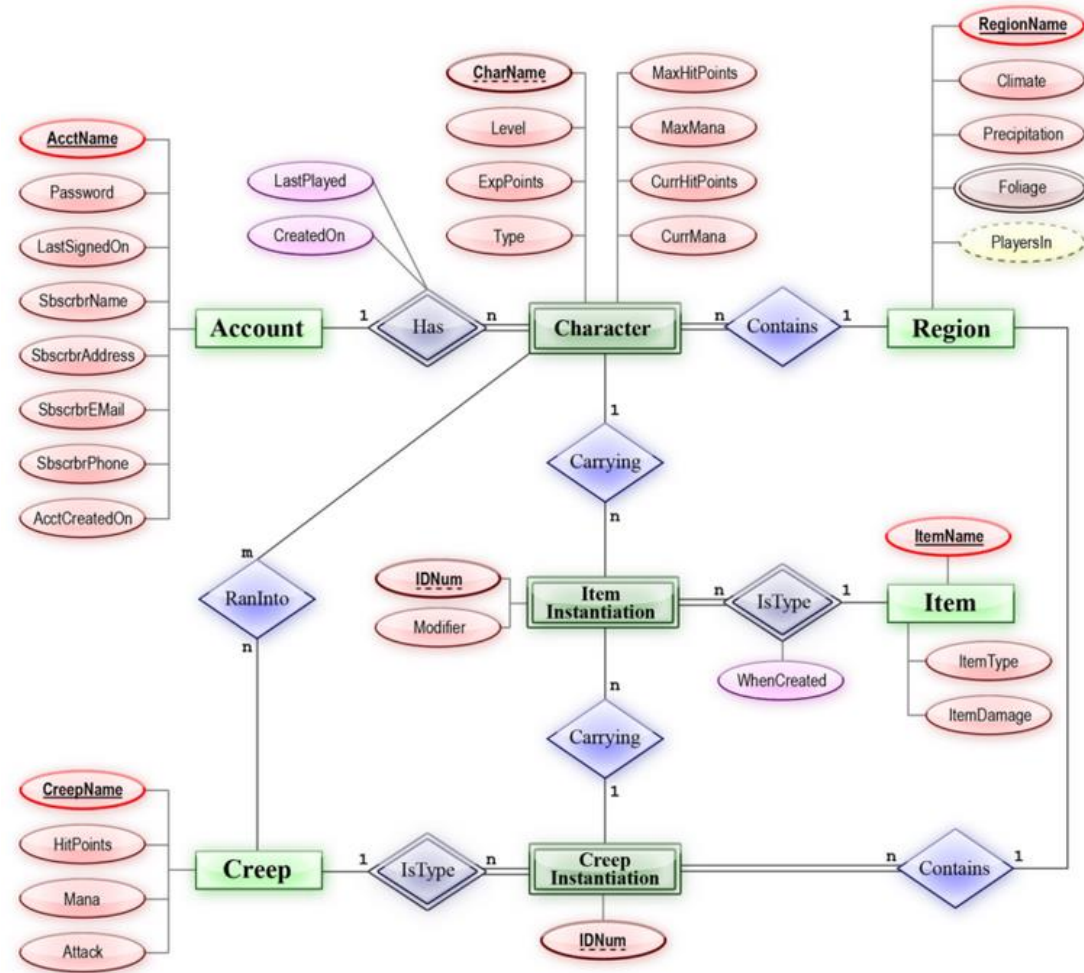
DATABASE SYSTEMS

CHAP. 2: Data Modeling Using the Entity-Relationship (ER) Model

Entity-Relationship Diagram (ERD)

- Graphical approach to database design.
- It is a high-level data model that defines data elements and their relationship for a specified software system.
- An ER model is used to represent real-world objects.

ERD



Entity

- Distinguishable object that exists.
- Thing in the real world with an independent existence.
- Example:
 - Company
 - Job
 - University
 - Course

Entity Type

- Defines a collection (or set) of entity that have same attributes.
- Each entity type in the database is described by its name and attributes.

Special Entity Types

- Associative entities
 - Aka intersection entities, are entities used to associate two or more entities in order to reconcile a many-to-many relationship.

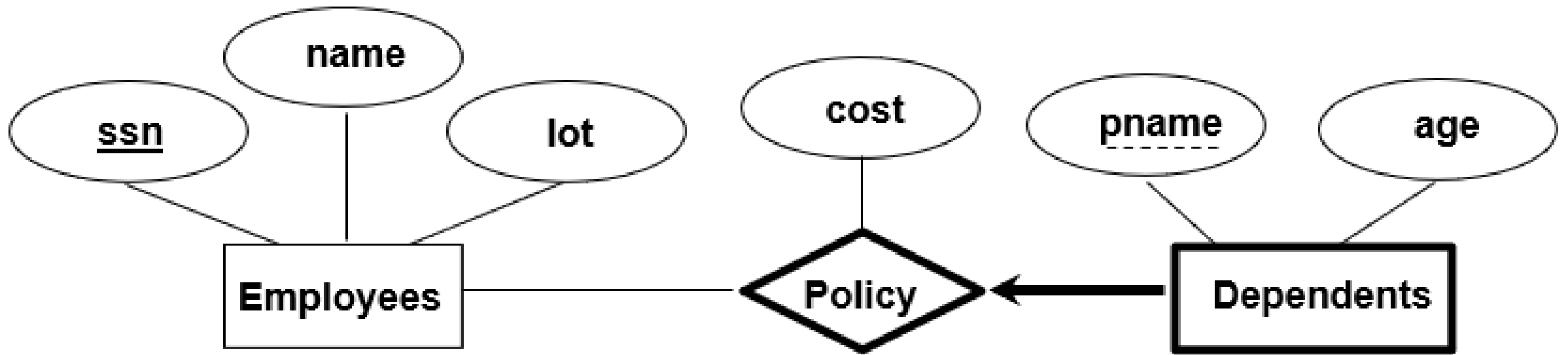
Special Entity Types

- Subtypes entities
 - They are used in generalization hierarchies to represent a subset of instances of their parent entity, called the supertype, but which have attributes or relationships that apply only to the subset.

Weak Entity

- A weak entity can be identified uniquely only by considering the primary key of another (owner) entity.
 - Owner entity set and weak entity set must participate in a one-to-many relationship set (one owner, many weak entities).
 - Weak entity set must have total participation in this identifying relationship set.

Weak Entity



Weak entities have only a “partial key” (dashed underline)

Strong Entity

- Exist independently of other types of entities
- Has its own unique identifier
- Represented with single-line rectangle

Entity set

- Set of entities of the same type that share the same properties.
- Example:
 - Set of all persons, companies, trees, holidays

Attribute

- Properties used to describe an entity
- Example for an EMPLOYEE entity
 - Attributes:
 - Name
 - SSN
 - Address
 - Sex
 - BirthDate

Types of Attributes

- Atomic Attribute
- Composite Attribute
- Single-Valued Attribute
- Multi Valued Attribute
- Stored Attribute
- Derived Attribute
- Prime Attribute
- Non-prime Attribute

Atomic Attribute

- An attribute that is not divisible and composed of a single component with an independent existence.
- Examples for the employee entity :
 - position
 - salary.
- Aka Simple Attributes

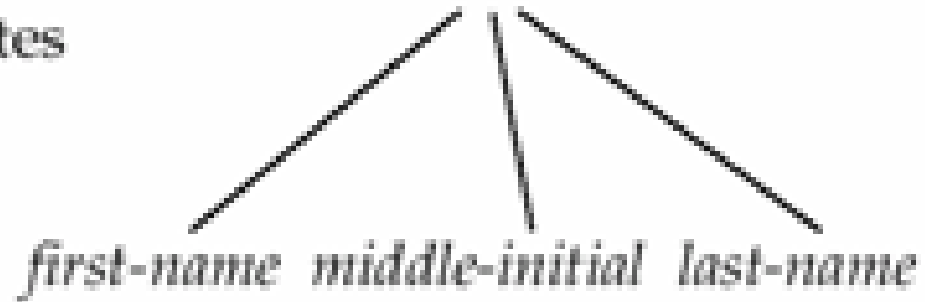
Composite Attribute

- An attribute that can be divided into smaller subparts, which represent more basic attributes with independent meanings.

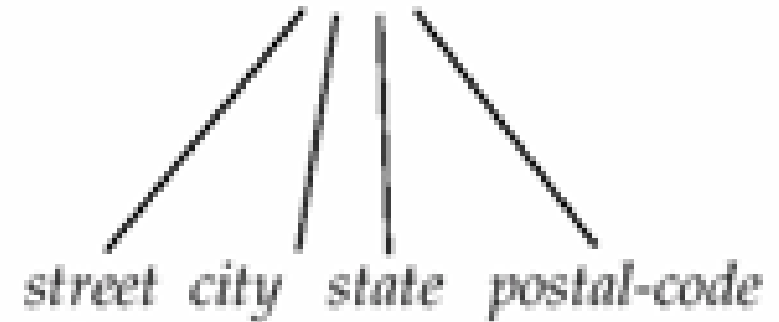
Composite Attribute

Composite
Attributes

name



address



Component
Attributes



Types of Attributes

- Single-Valued Attribute
 - An attribute that has a single value for a particular entity.
- Multi Valued Attribute
 - An attribute that has multiple values for a particular entity.
- Stored Attribute
 - An attribute value that is stored in a table.

Types of Attributes

- **Derived Attribute**
 - An attribute that represents a value that is derivable from the value of a related attribute or set of attributes, not necessarily in the same entity type.
 - Or it can be computed from other attributes.
- **Example:**
 - age, given date of birth.
- **Prime Attribute**
 - An attribute which participates in at least one key.
- **Non-prime Attribute**
 - All other attributes which don't participate in a key are non-prime

Types of Attributes

- **Keys**

- Attribute whose value uniquely identifies an entity of that type.
- No two rows have the same key.
- Attribute utilized to sort.

Types of Attributes

- **Candidate key (never NULL)**

- Minimal set of attributes that uniquely identifies each occurrence of an entity type.
- Example for in entity branch:
 - Branchno

- **Primary key**

- Candidate key that is selected to uniquely identify each occurrence of an entity type.
- Example:
 - National insurance number.

- **Alternate key**

- Candidate keys which are not currently selected as the primary key.

Types of Attributes

•Foreign Key

- utilized to cross-reference data between relational tables.
- primary key from a particular table that is stored in a second or related table.
- attribute value for one relation is drawn from another relation.

•Composite Key

- candidate key that consist of two or more attributes.
- Aka Compound key.

Types of Attributes

• Superkey

- A superkey is a set of attributes that has at least as much information that is necessary to uniquely identify a records.
- Every relation has at least one superkey which is the set of all attributes.
- Superkeys can have redundant attributes whereas a key has no redundancy.

Types of Attributes

•Surrogate key

- A surrogate key is a single-part, artificially established identifier for an entity.
- Surrogate key assignment is a special case of derived data - one where the primary key is derived.
- Common way of deriving surrogate key values is to assign integer values sequentially
- A surrogate key is a unique, DBMS-supplied identifier used as the primary key of a relation
- The values of a surrogate key have no meaning to the users and are normally hidden on forms and reports
- DBMS does not allow the value of a surrogate key to be changed

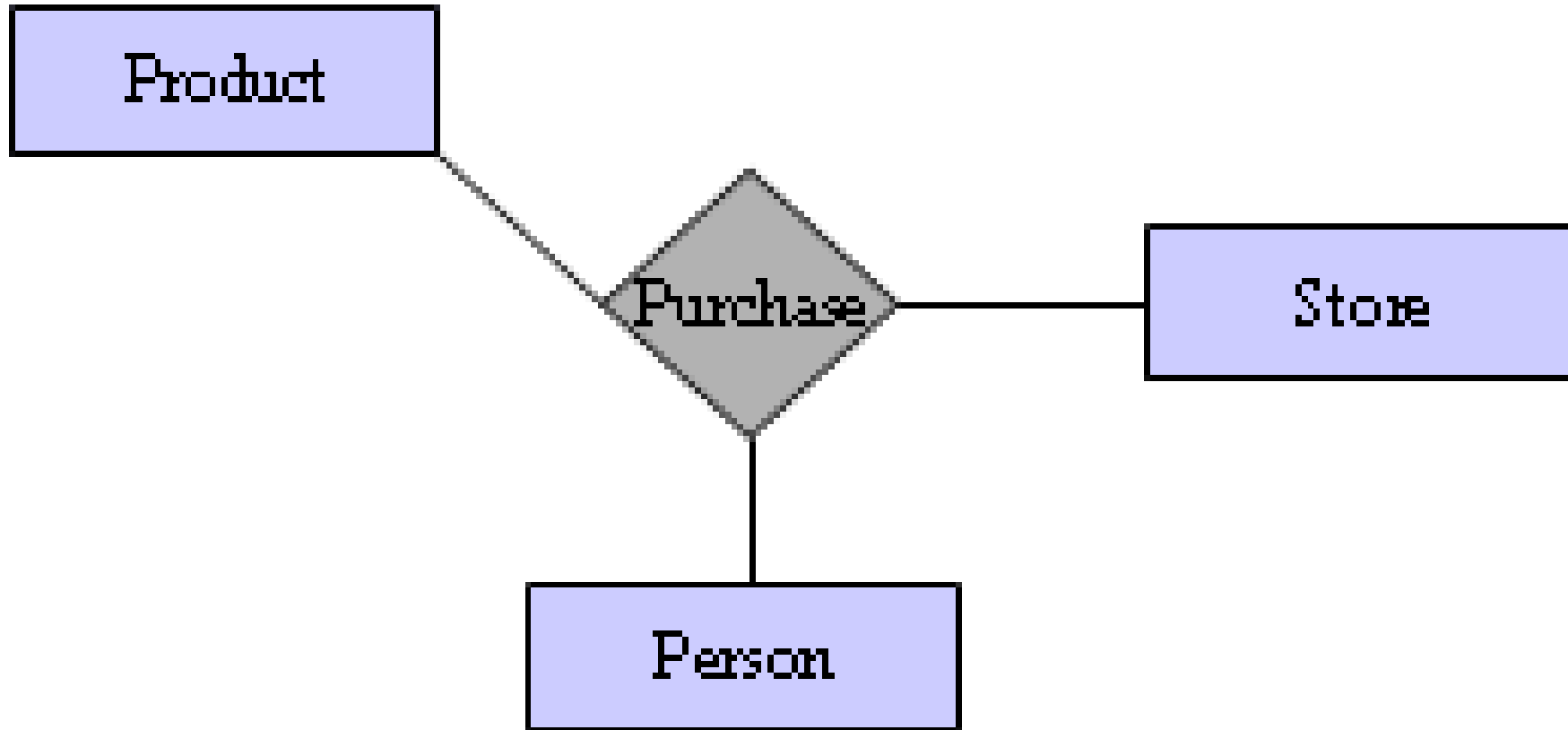
Relationship

- Association among two or more entities.
- Occur between entity types.
- Attribute of one entity type *refers* to another entity type.
- Examples:
 - Kabeba ***works*** in Pharmacy department.
 - Maniraguha ***teaches*** in the Math & CS Dept.

Relationship degree

- Number of participating entity types.
- Binary
 - Employee works_for department.
- Ternary
 - Students take classes during particular semesters.
- N-ary
 - represents more information than three binary relationships.

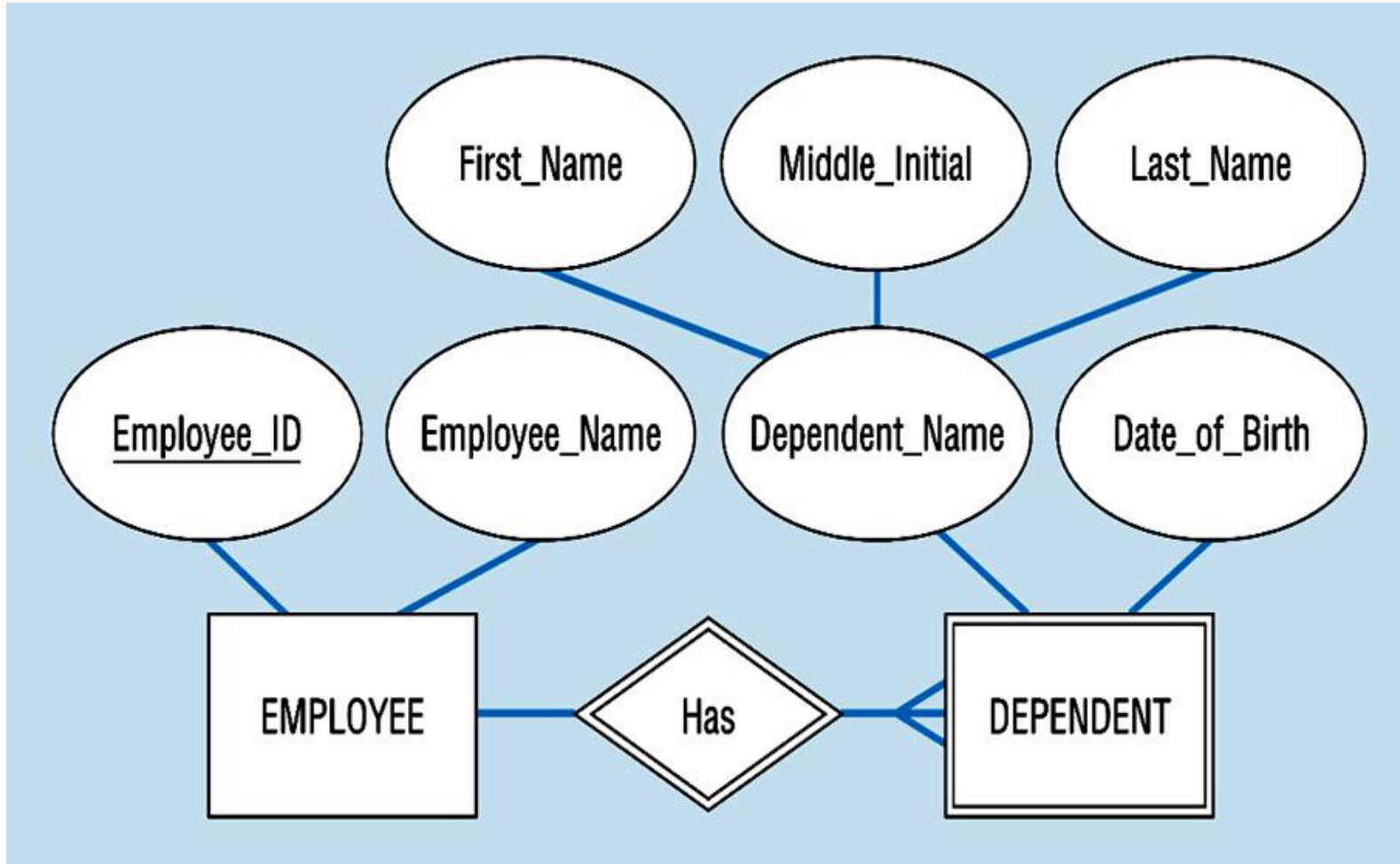
Relationship degree



Identifying Relationship

- Links strong entities to weak entities
- Represented with double line diamond

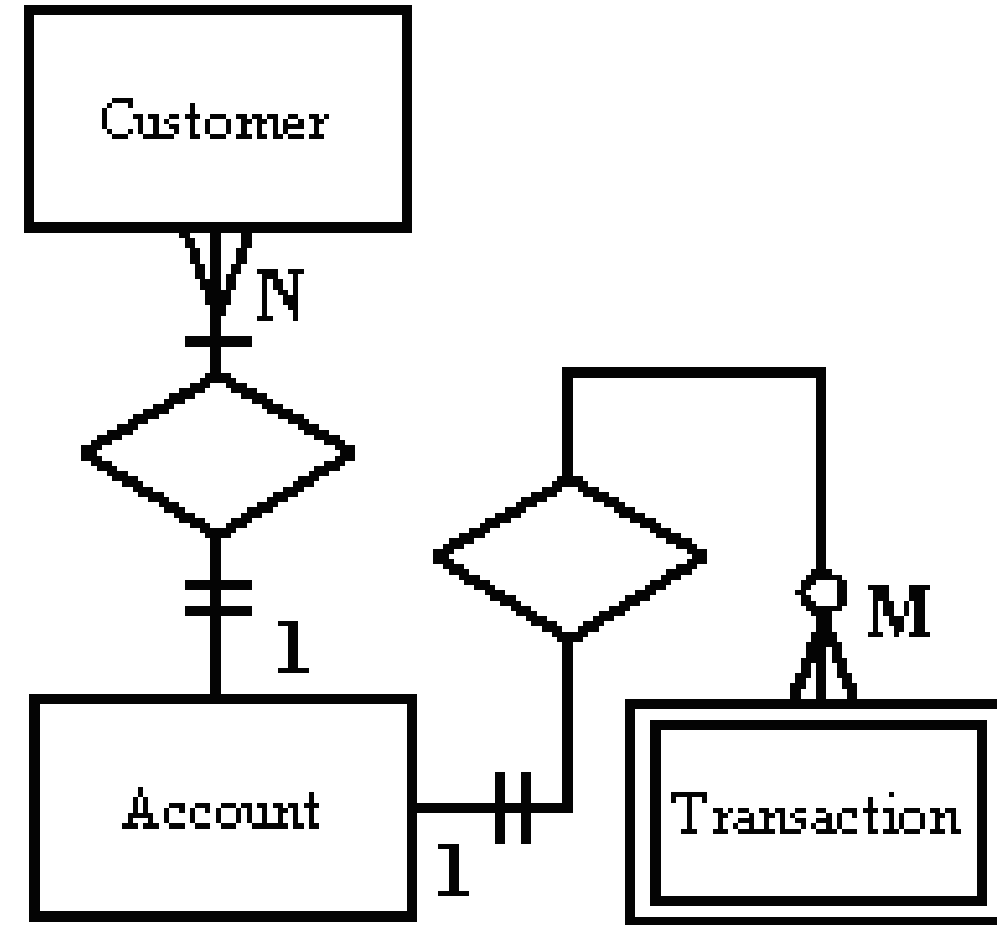
Strong, weak entities



Cardinality

- Specifies how many instances of an entity relate to one instance of another entity.

Cardinality



Cardinality

Ordinality

- Describes the relationship as either mandatory or optional.
- Specifies the maximum number of relationships and ordinality specifies the absolute minimum number of relationships.

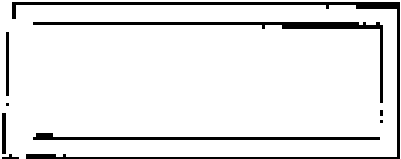
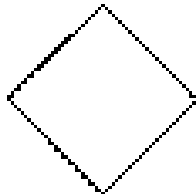
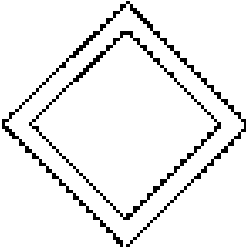
Type of Ordinalities

- One to one (1:1)
- One to many (1:N)
- Many to one (N:1)
- Many to many (N:M)

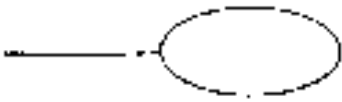
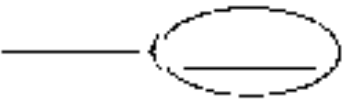
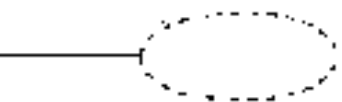
ERD Notation

- **rectangles**: represent entity sets.
- **ellipses**: represent attributes.
- **diamonds**: represent relationships among entity sets.
- **lines**: link attributes to entity sets and entity sets to relationships.

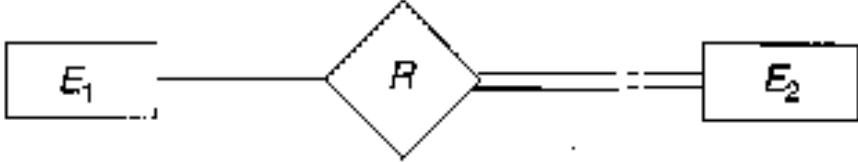
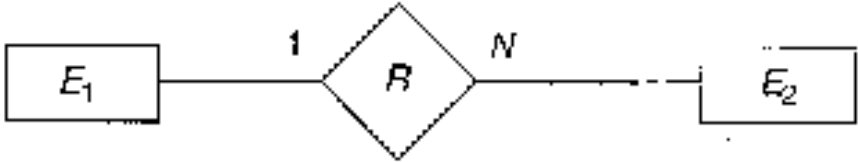
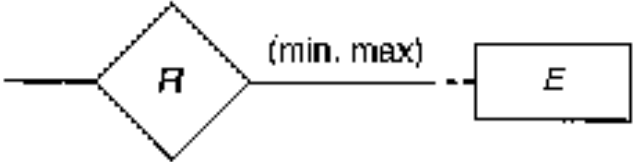
ERD Notation

<u>Symbol</u>	<u>Meaning</u>
	ENTITY
	WEAK ENTITY
	RELATIONSHIP
	IDENTIFYING RELATIONSHIP

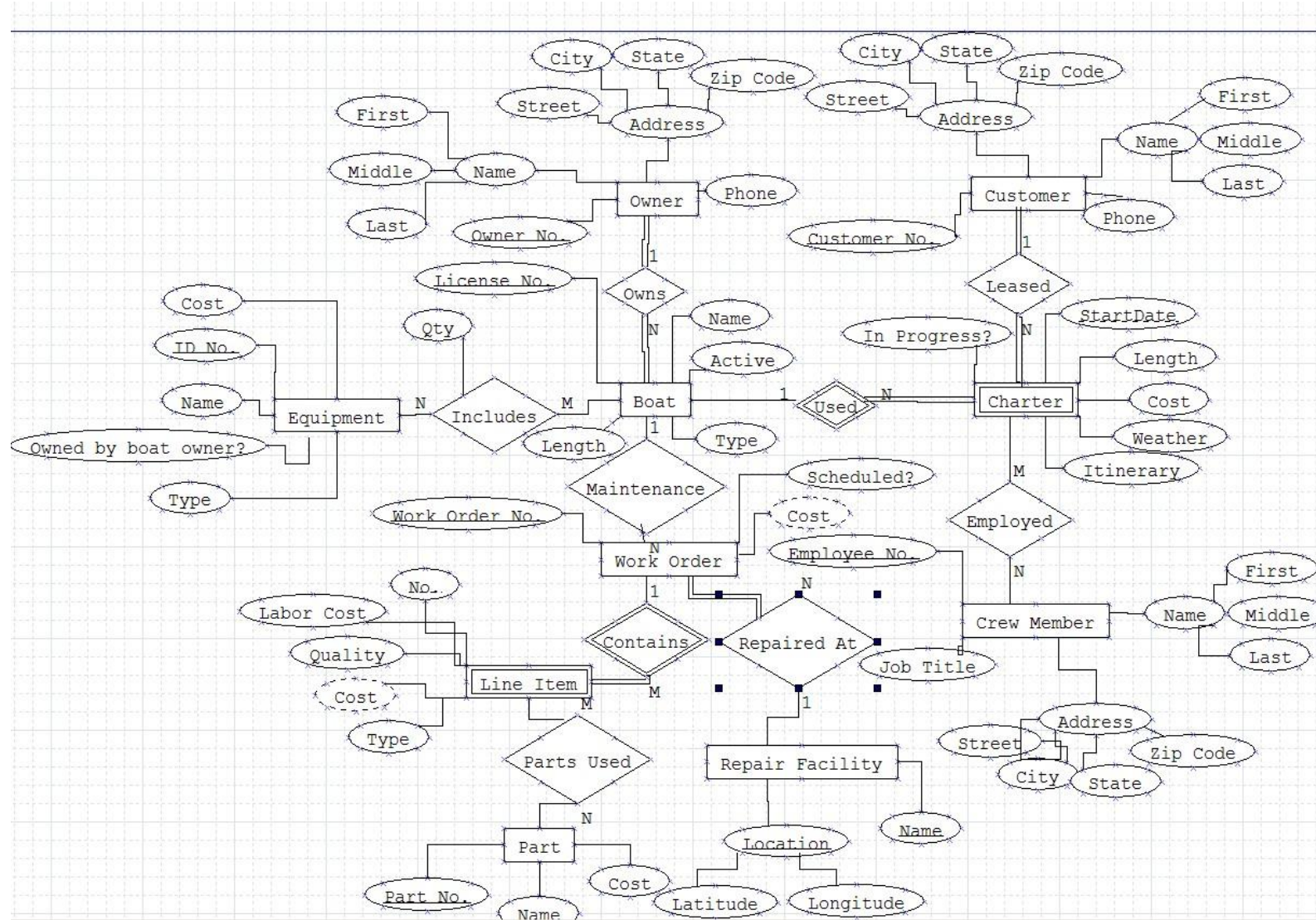
ERD Notation

<u>Symbol</u>	<u>Meaning</u>
	ATTRIBUTE
	KEY ATTRIBUTE
	MULTIVALUED
	COMPOSITE ATTRIBUTE
	DERIVED ATTRIBUTE

ERD Notation

Symbol	Meaning
	TOTAL PARTICIPATION OF E_2 IN R
	CARDINALITY RATIO 1: N FOR E_1 : E_2 IN R
	STRUCTURAL CONSTRAINT (min, max) ON PARTICIPATION OF E IN R

ERD



ER-to-Relational Mapping

- **Mapping of strong entity types**
 - Each strong entity set becomes a (table)
 - Each attribute becomes a column in the table
 - Identifying attribute becomes the primary key.
 - Name given to the relation is generally the same as the entity type

Mapping of Weak Entity Types

- For each weak entity type, create a new table and include all of the simple attributes (or simple components of composite attributes) as attributes of this table
- Then include the primary key of the identifying table as a foreign key attribute in this new table
- The primary key of the new table is the combination of this primary key of the identifying table and the partial identifier of the weak entity type

Mapping of N-ary Relationship Types

- For each n -ary relationship type R , where $n > 2$, create a new table S to represent R .
- Include as foreign key attributes in S the primary keys of the tables that represent the participating entity types.
- Also include any simple attributes of the n -ary relationship type as attributes of S .

Mapping of Binary M:N Relationship Types

- If such a relationship exists between entity types A and B, we create a new relation C, then include as foreign keys in C the primary keys for A and B, then these attributes become the primary key of C

Mapping of Binary 1:1 Relation Types

- These can be viewed as a special case of one-to-many relationships. Firstly, two relations are created, one for each of the participating entity types
- primary key of one of the relations is included as a foreign key in the other relation
- You should include in the relation on the optional side of the relationship the foreign key of the entity type that has the mandatory participation in the 1:1 relationship.

Mapping of Binary 1:N Relationship Types

- First create a relation for each of the two entity types participating in the relationship
- Next include the primary key attribute(s) of the entity on the one-side as a foreign key in the relation that is on the many-side.

Mapping of Multivalued attributes

- Here two new relations (rather than one) are created
- First relation contains all of the attributes of the entity type except the multivalued attribute
- Second relation contains two attributes that form the primary key of the second relation
- The first of these is the primary key for the first relation, which becomes a foreign key in the second relation
- The second is the multivalued attribute.

Mapping of Composite attributes

- When a regular entity type has composite attributes, only the simple component attributes of the composite attribute are included in the new relation

Mapping of high-level entities

- Each high-level entity set becomes a relation
- Each attribute becomes a column
- If it has its own identifier as primary key
- Primary keys of its associated entity sets as non-key columns
- If it has not its own identifier, follow the same procedures as relationship sets

(Optional) Combine/Spilt Tables

- Remove one-column tables
- Use referential integrity instead
- Combine tables with the same primary key
- Be careful because it might result in lower normal form
- Spilt a table into multiple tables, to maximize the operational efficiency

Database integrity

- Ensures that data entered into the database is accurate, valid, and consistent.
- Any applicable integrity constraints and data validation rules must be satisfied before permitting a change to the database.

Data integrity

- Enforcing data integrity guarantees the quality of the data in the database.

Data integrity(Cont.)

- Example:
 - If an employee is entered with an employee ID value of 123, the database should not permit another employee to have an ID with the same value.

Data integrity(Cont.)

- Example:
 - If you have an employee_rating column intended to have values ranging from 1 to 5, the database should not accept a value outside that range.

Data integrity(Cont.)

- Example:
 - If the table has a dept_id column that stores the department number for the employee, the database should permit only values that are valid for the department numbers in the company.

Referential Integrity

- Ensures that relationships between tables remain consistent.
- When one table has a foreign key to another table, the concept of referential integrity states that you may not add a record to the table that contains the foreign key unless there is a corresponding record in the linked table.
- Includes the techniques known as cascading update and cascading delete, which ensure that changes made to the linked table are reflected in the primary table.

Entity Integrity

- ensures that no records are duplicated and that no attributes that make up the primary key are NULL.
- It is one of the properties necessary to ensure the consistency of the database.
- The existence of the Primary Key is the core of the entity integrity.
- If you define a primary key for each entity, they follow the entity integrity rule.

Domain Integrity

- Validity of entries for a specific column.
- You can enforce domain integrity to restrict the type by using data types, restrict the format by using check constraints and rules, or restrict the range of possible values by using foreign key constraints, check constraints, default definitions, not null definitions, and rules.

User-Defined Integrity

- User-defined integrity lets you define specific business rules that do not fall into one of the other integrity categories.
- All the integrity categories support user-defined integrity.
- This includes all column-level and table-level constraints in CREATE TABLE, stored procedures, and triggers.